

Foundational Principles for understanding SE

Anil S. Mokhade 21, 22,23 Jan 2026

Introduction

"To master Software Engineering, you must first understand its foundational principles. These principles will shape how you think about designing, building, and maintaining software systems. Today, we'll explore three essential pillars:

Concepts, Strategies, and Operations, which form the framework for understanding Software Engineering."

Objective:

This lecture introduces you to a structured approach to learning Software Engineering by defining and applying the three foundational elements:

- **Concepts**
- **Strategies**
- **Operations**

This lecture will help you build a mindset to approach Software Engineering holistically.

Introduction

"Software Engineering is not just about writing code. It's about solving problems systematically, using concepts, strategies, and operations that ensure software is reliable, maintainable, and scalable."

- Welcome to the course
- Let us start with an engaging thought:
"The best engineers are not the ones who know everything, but those who know how to think systematically."
- **Goal:** is in understanding **Concepts, Strategies, and Operations** will help us effectively learn and apply Software Engineering principles.

Defining Concepts, Strategies, and Operations

🔑 1. What is a Concept?

A **Concept** is a core idea or principle that forms the foundation of a subject.

In Software Engineering, concepts are the building blocks that we use to structure our understanding of how software is built, maintained, and improved.

✅ Examples of Key Concepts in Software Engineering:

- **Modularity**: Breaking down a system into smaller parts.
- **Abstraction**: Hiding the complexity of systems to simplify interaction.
- **Software Lifecycle**: Different stages from development to maintenance.

Abstraction: Hiding Complexity

Abstraction is about **deciding what to hide and what to expose**.

- Focuses on **essential behavior**
- Suppresses irrelevant implementation details
- Enables reasoning at higher levels

Example abstractions:

- Function signatures
- Abstract data types (ADTs)
- Interfaces / APIs
- Virtual machines
- Services in SOA / microservices

Abstraction answers: "What does this do?"

Modularity: Structuring Complexity

Modularity is about **how software is decomposed into parts**.

- Separates concerns
- Enables independent development
- Enables independent testing and evolution
- Localizes change

Example modules:

- Classes and packages
- Libraries
- Services
- Components
- Plugins

Modularity answers: "Where does this belong?"

How Abstraction Enables Modularity

A **module exists because an abstraction exists**.

- You **cannot modularize what you cannot abstract**
- Each module **embodies one or more abstractions**
- Interfaces are **abstraction boundaries**
- Implementation is **hidden behind module boundaries**

Classical Principle: Information Hiding

Introduced by **David Parnas**:

Modules should be designed around design decisions that are likely to change.

This is the **bridge** between abstraction and modularity:

- **Abstraction** hides the change-prone decisions
- **Modularity** physically isolates them

How Modularity Preserves Abstraction Over Time

Without modularity:

- Abstractions **leak**
- Dependencies **entangle**
- Systems **rot**

With strong modularity:

- Abstractions remain stable
- Implementations can change freely
- Systems evolve without collapse

This insight is central to **Frederick P. Brooks Jr.**'s observation:

The hardest single part of building a software system is deciding what to build — i.e., defining abstractions.

Examples of Abstraction and Modularity:

Operating Systems

- **Abstraction:** Process, file, virtual memory
- **Modularity:** Kernel modules, drivers, subsystems

Unix succeeded because:

- Small, stable abstractions
- Clear module boundaries

Microservices

- **Abstraction:** Service contracts (APIs)
 - **Modularity:** Independently deployable services
- When abstraction is weak:
- “Distributed monoliths” emerge

So, What we can say about this Relationship between Abstraction & Modularity:

Abstraction defines the intellectual boundary; modularity enforces it structurally.

Abstraction manages **cognitive complexity**, while modularity manages **organizational and evolutionary complexity**.

Why This Relationship Is Fundamental to Software Engineering

From a **Software Engineering (not programming)** viewpoint:

They together enable:

- **Scalability**
- **Team collaboration**
- **Long-term maintainability**
- **Economic viability of software products**

Aspect	Abstraction	Modularity
Primary role	Reduce complexity	Localize complexity
Focus	Conceptual	Structural
Enables	Understanding	Evolution
Failure mode	Leaky abstractions	Spaghetti dependencies
Engineering concern	Correctness	Maintainability

So, What we can conclude from this relationship between Abstraction & Modularity:
Software Engineering exists because abstraction alone is not enough —
and modularity alone is meaningless without abstraction.

🌱 2. What is a Strategy?

A **Strategy** is a method or plan of action to achieve a goal.

In Software Engineering, strategies are **approaches we use to design, build, test, and maintain** software systems effectively.

✅ Examples of Strategies in Software Engineering:

- **Agile Development:** Iterative and incremental software development methodology.
- **Design Patterns:** Reusable solutions to common software design problems.
- **Testing Strategies:** Approaches to ensure software works as expected (e.g., unit testing, integration testing).

🔧 3. What is an Operation?

An **Operation** is a concrete action or task that applies a strategy to a specific problem.

Operations are the **day-to-day tasks** in software engineering, such as writing code, debugging, testing, refactoring, etc.

✅ Examples of Operations in Software Engineering:

- Writing a function to implement a feature.
- Refactoring a module to improve performance.
- Running test cases to verify correctness.

Application to Software Engineering

🏠 Building Blocks Example:

Imagine you are building a **House Software System**.

Level	Example in Software Engineering	Example from Real Life
Concept	Modularity	The idea of separating rooms in a house.
Strategy	Layered Architecture	Designing a multi-floor house.
Operation	Writing Code for Room Heating System	Installing heating systems.

Connecting to Software Engineering Topics:

- **Concepts:** Software lifecycle, abstraction, modularity.
- **Strategies:** Agile development, testing strategies, DevOps practices.
- **Operations:** Writing code, debugging, deployment.

“Engineering is not just about building things; it's about building the right things the right way.”

Thank You

CSL308 Software Engineering

Concepts, Strategies, and Operations Framework for understanding Software Development - Domain Examples Domain Concept

Anil S. Mokhade 3,4, 6 Feb 2026

“If software is wrong in an e-commerce app, we lose money.
If software is wrong in a hospital, what do we lose?”

“That is why domain knowledge and software development process cannot be separated.”

**Software processes were not invented in textbooks.
They were invented after things went wrong.**

Let us consider the case of a campus hospital automation.

A **campus hospital** serves a unique and highly dynamic population comprising **students, faculty, staff, and visitors**, with wide variations in age, medical history, and service needs. Unlike large public hospitals, a campus hospital must operate with **limited resources, high patient turnover**, and **time-critical decision making**, especially during academic sessions, examinations, and emergencies. In this context, a **dedicated software system** becomes essential rather than optional.

The primary need for a campus hospital software system lies in **efficient patient information management**. Students and staff often visit repeatedly for follow-ups, chronic conditions, vaccinations, or medical fitness certificates. A centralized **electronic health record (EHR)** ensures continuity of care, reduces dependency on paper files, and enables doctors to make informed decisions quickly. It also minimizes errors caused by missing or incomplete medical histories.

Secondly, the software system supports **operational efficiency and coordination**. Modules for registration, appointment scheduling, pharmacy, laboratory reports, and inventory management help streamline daily workflows. This reduces waiting time, optimizes the utilization of doctors and nurses, and ensures that medicines and consumables are available when needed—an important requirement in a campus environment where sudden health outbreaks or seasonal illnesses are common.

A campus hospital software system is also critical for **emergency preparedness and safety**.

Integration with campus systems such as student databases, ID cards, and emergency contacts allows rapid identification of patients and timely communication with authorities or family members. In institutions with hostels, sports facilities, and laboratories, such integration can be lifesaving during accidents or medical emergencies.

Finally, the software system enables **administrative control, compliance, and data-driven planning**.

Accurate digital records support statutory reporting, medical audits, insurance claims (where applicable), and health analytics. Over time, aggregated data can help campus authorities identify health trends, plan preventive health programs, and allocate resources more effectively.

In essence, a software system for a campus hospital transforms healthcare delivery from a **reactive, manual process** into a **proactive, reliable, and integrated service**, aligning healthcare operations with the academic institution's responsibility for safety, well-being, and institutional resilience.

Software systems don't exist in isolation — they exist to solve real-world problems in specific domains like healthcare, education, finance, or e-commerce. These domains have their own **concepts**, rules, and workflows. As software engineers, our first task is to **identify and understand these domain concepts**, because they form the **foundation** of the system we design.

Domain knowledge grows faster than code.

Characteristics of Domain Concepts:

1.Problem-Specific:

Domain concepts are **specific to the problem space** or industry (e.g., books in a library system, patients in a medical diagnosis system).

2.Real-World Entities and Processes:

They represent **real-world entities, rules, and workflows** that the software system must capture and support (e.g., orders, payments, reservations).

3.Independent of Technology:

Domain concepts are **independent of programming languages or frameworks**. They are focused on **what the system needs to represent and handle** rather than how it is implemented.

4.Key to Requirements Gathering:

Understanding domain concepts is essential during **requirements engineering**. They help translate **user requirements into technical solutions**.

1. Real-World Problems Are Domain-Specific

Every **real-world problem** comes from a specific **domain** with **distinct rules and workflows**.

For example:

- In a **hospital management system**, key domain concepts include **patients, doctors, appointments, and prescriptions**.
- In a **university system**, domain concepts include **students, courses, exams, and grades**.

If a software engineer doesn't understand these **domain-specific concepts**, they will struggle to design software that accurately represents the real world.

Why Real-World Software Problems Demand Domain Concept Understanding from Software Engineers?

Software engineers must understand **domain concepts** because **real-world problems** exist within **specific domains** (e.g., education, healthcare, finance) that have **unique entities, processes, and rules**. Without a clear grasp of these domain concepts, software solutions may fail to solve the **right problems** or meet **stakeholder needs**.

🔍 Example: Why Domain Knowledge Matters in a Hospital System

Scenario	Without Domain Knowledge	With Domain Knowledge
Appointment Scheduling	Might allow conflicting appointments	Ensures that doctor availability and patient history are considered
Prescription Management	May not account for drug interactions	Includes drug interaction checks and patient allergies
Patient Records	Stores incomplete or irrelevant data	Collects complete, domain-specific patient information

🔑 Key Insight:

Without understanding the **domain concepts**, software engineers will create **flawed or incomplete solutions** that don't align with **real-world requirements**.

🧑 2. Domain Concepts Are Key to Stakeholder Communication

Software engineers need to **understand domain concepts** to communicate effectively with **stakeholders** and translate their needs into technical requirements.

Stakeholders (e.g., students, doctors, administrators) are **domain experts**, not software experts. They speak in terms of **real-world concepts** (like patients, courses, or products), not technical jargon.

🔄 How Domain Concepts Improve Communication

Without Domain Understanding	With Domain Understanding
Misunderstands stakeholder needs	Accurately captures stakeholder requirements
Uses technical jargon	Speaks the language of the domain
Misses critical business rules	Identifies and implements business rules correctly

⚙️ 4. Domain Concepts Influence Software Design

Once domain concepts are identified, they **influence the software design** by shaping:

- **Data Models:** How data is stored and organized.
- **Business Logic:** The rules and workflows the system follows.
- **User Interfaces:** How users interact with the system.

🔑 Example: Designing a University System

Domain Concept	Software Design Impact
Students	Design a student database table with relevant attributes (name, ID, grades).
Courses	Design a course module with registration rules.
Grades	Implement grading logic and reporting dashboards.

Without understanding domain concepts, software engineers may design **data models and workflows that don't match the real-world domain**, leading to **poor usability and functionality**.

💡 Example: Course Registration System

Stakeholder: **University Registrar**

- They talk about **students enrolling in courses**, **course prerequisites**, and **class schedules**.

Software Engineer:

- Without domain knowledge: May focus on **database tables** and **API endpoints** without understanding the **business rules** of course registration.
- With domain knowledge: Understands concepts like **course capacity**, **pre-requisite checks**, and **drop deadlines**.

🔑 Key Insight:

Understanding **domain concepts** enables software engineers to **speak the language of stakeholders**, ensuring that **requirements are well-defined and accurate**.

📌 3. Domain Concepts Ensure Accurate Requirements Engineering

Requirements engineering is the process of **gathering, analyzing, and documenting what a software system should do**. Domain concepts are at the heart of this process.

If software engineers fail to understand the domain concepts, they will likely gather **incomplete, vague, or incorrect requirements**, leading to **software failures**.

🔍 Common Issues Without Domain Understanding:

Problem	Description	Impact
Missing Requirements	Engineers overlook key domain rules	The system doesn't handle critical scenarios
Incorrect Requirements	Engineers misunderstand domain processes	The system behaves incorrectly
Ambiguous Requirements	Engineers capture vague or unclear requirements	The system doesn't meet user expectations

📌 Key Insight:

Accurate **requirements engineering** starts with understanding **domain concepts** so that engineers can capture **complete, relevant, and correct requirements**.

🧠 5. Domain Concepts Improve Software Reusability

Understanding domain concepts also enables **Domain Engineering**, where software engineers create **reusable components** that can be used across multiple systems in the same domain.

For example:

- In an **academic domain**, reusable components could include **student management modules**, **course registration systems**, and **grading tools**.
- In a **healthcare domain**, reusable components might include **patient record systems**, **appointment schedulers**, and **billing systems**.

By understanding the **commonalities and variabilities** in a domain, engineers can **design reusable solutions**, reducing development time and improving quality.

📊 Summary of Key Reasons:

Reason	Description	Example
Real-World Problems Are Domain-Specific	Domain concepts reflect real-world entities and rules	Hospital systems need to handle patients, doctors, prescriptions
Improves Stakeholder Communication	Helps engineers speak the language of stakeholders	Faculty talk about courses and grades, not tables and APIs
Ensures Accurate Requirements	Captures complete and relevant requirements	Course registration system must account for prerequisites
Influences Software Design	Shapes data models, business logic, and UI	Design a student management system with accurate data models
Improves Reusability	Enables creation of reusable components	Build reusable academic system modules for different universities

🌟 Final Thought: Why Software Engineers Must Understand Domain Concepts

"Software is not built in isolation. It must solve **real-world problems** in **specific domains**, which have their own entities, rules, and processes. **Understanding domain concepts** ensures that software engineers can **communicate effectively with stakeholders**, **gather accurate requirements**, **design relevant solutions**, and **reuse components across systems**. Without domain knowledge, software may fail to address the actual problems it's meant to solve."

... Example to Reinforce the Transition:

Library Management System

Let's revisit the **Library Management System** example from earlier:

Domain Concept	Software Concept
Books	Database Design
Borrowing Rules	Business Logic
Users (Students/Staff)	Authentication and Access Control

Notice how the real-world entities in the library system — books, users, and borrowing rules — are **domain concepts**. These domain concepts drive the software concepts we apply, such as database design and authentication.

Can you identify the domain concepts in some of the other examples we discussed, like the e-commerce platform or the flight simulator?

In the next section, we'll dive deeper into the importance of **domain concepts**, how they influence software design, and how they help bridge the gap between **real-world problems** and **technical solutions** in Software Engineering.

🌱 Defining Domain Concepts in Software Engineering

In Software Engineering, **domain concepts** refer to the **core ideas, entities, and processes specific to the real-world problem domain** that the software system is intended to address.

These concepts **do not belong to software itself**, but rather to the **subject area or business context** for which the software is being developed. Understanding domain concepts is **crucial for accurately modeling and designing software systems** that meet user needs.

💡 Difference Between Domain Concepts and Software Concepts:

Aspect	Domain Concepts	Software Concepts
Definition	Real-world entities and processes	Technical principles and constructs
Example	Books, Patients, Products, Payments	Modularity, Abstraction, Error Handling
Focus	What the system needs to represent	How the system is designed and implemented
Purpose	Captures business needs and workflows	Ensures efficient, maintainable system design

🔗 Why Domain Concepts Are Important in Software Engineering:

1.Accurate Problem Understanding:

They help developers understand the **real-world problem** they are solving.

2.Effective System Design:

Domain concepts guide the creation of **models** such as **UML diagrams**, **ER diagrams**, and **domain models** that reflect the **real-world structure** of the problem.

3.Communication Bridge:

They provide a **common language** for communication between **stakeholders** (e.g., clients, users) and **developers**.

📊 Examples of Domain Concepts in Different Software Systems:

Domain	Domain Concepts	Software Concepts Used
Library System	Books, Users, Borrowing Rules	Database Design, User Authentication
E-Commerce Platform	Products, Orders, Payments	Microservices Architecture, Transaction Management
Medical Diagnosis	Patients, Symptoms, Diagnosis	Machine Learning, Data Security
Flight Simulator	Aircraft, Weather Conditions, Controls	Real-Time Processing, Physics Engines
Stock Market Tool	Stocks, Trends, Risk Analysis	Data Analysis, Visualization

🔗 Connecting Domain Concepts to CSO Framework:

Domain concepts are the **inputs** into the **Concept-Strategy-Operation** framework.

- **Concepts** in CSO are derived from **domain concepts** (e.g., "Books" becomes part of the "Database Design" concept).
- **Strategies** are developed to handle domain concepts effectively (e.g., a relational database to store "Books" and "Users").
- **Operations** implement domain concepts in practice (e.g., CRUD operations to manage "Books").

📖 CRUD Operations in Software Engineering

CRUD stands for **Create, Read, Update, and Delete** — the **four basic operations** used to interact with and manage data in a software system. These operations are fundamental to designing and building any application that works with **persistent data** (data stored in databases, files, or other storage systems).

"At the core of every software system lies the **problem domain** it is designed to solve. Understanding the **domain concepts** is the first step in building meaningful software that aligns with user needs. These concepts guide the way we think about **what the system should do**, and they shape the **software concepts, strategies, and operations** we use to implement the solution."

🔑 **Key Insight:**

Domain concepts are not about technology — they are about people, processes, and real-world entities. By understanding domain concepts, we can bridge the gap between **the real world** and **software design**, ensuring that the systems we build are not only technically sound but also **relevant, useful, and impactful**.

🎯 **Why This Matters:**

"Great software doesn't start with code — it starts with understanding the world it's meant to improve. As software engineers, our task is to model that world accurately through domain concepts and translate it into efficient, maintainable, and scalable software solutions."

👤 **Next Step:**

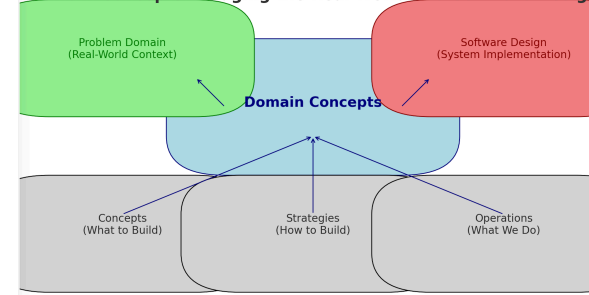
"Now that we understand **domain concepts**, let's explore how these concepts influence different types of software systems. We'll also see how the **Concept-Strategy-Operation (CSO)** framework applies across these types to give us a complete picture of the Software Engineering process."

But before that :

"Now that we've explored **domain concepts** and how they define the problem space, the next step is to understand **who the system is being built for** and **what they need** from it. These people, known as **stakeholders**, are essential to shaping the system's **requirements**, which form the foundation for the entire **software development process**."

This we will explore in the next lecture.

Domain Concepts: Bridging the Real World and Software Design



Domain Concepts bridge the gap between **Real-World Problem Domains** and **Software Design**. The diagram also shows how the **Concept-Strategy-Operation (CSO)** framework flows from understanding domain concepts to building software solutions.

Thank You

1. Domain Concept & Stakeholders

2. Software Processes

Anil S. Mokhade 21 Jan 2025

1. Domain Concept & Stakeholders

From Last Lecture

Next Step:

*"Now that we understand **domain concepts**, let's explore how these concepts influence different types of software systems. We'll also see how the **Concept-Strategy-Operation (CSO)** framework applies across these types to give us a complete picture of the Software Engineering process."*

But before that :

*"Now that we've explored **domain concepts** and how they define the problem space, the next step is to understand **who the system is being built for** and **what they need** from it. These people, known as **stakeholders**, are essential to shaping the system's **requirements**, which form the foundation for the entire **software development process**."*

This we will explore in the next lecture.

"We've seen how understanding the **domain concepts** of a system helps us model the real-world problem space accurately. But software doesn't exist in isolation — it exists to serve **people and organizations**. To build meaningful software, we need to understand **who the stakeholders are** and **what they expect from the system**. This is where the concepts of **stakeholders** and **requirements** come in."

💡 What Are Stakeholders?

Stakeholders are the **people, groups, or organizations** that have an **interest in the software system**. They influence the **requirements, design, and outcome** of the system.

📖 Types of Stakeholders:

Type of Stakeholder	Role	Example
End Users	People who use the system directly	Students using a university portal
System Owners	People who commission the system	University administration
Domain Experts	People with domain knowledge	Faculty providing input on courses
Regulators	People ensuring compliance with standards	Government bodies, accreditation agencies
Developers	People who build and maintain the system	Software engineers, project managers

📌 Why Are Stakeholders Important in Software Development?

1. **They define the system's purpose.**
 - The system must meet **stakeholder goals**.
2. **They provide the requirements.**
 - Stakeholders specify **what the system should do**.
3. **They ensure the system is usable.**
 - The system must be designed to meet stakeholder expectations.

Impact of Stakeholders on Software Development

1. **Requirements Gathering and Prioritization:**
 - Stakeholders provide critical input for functional and non-functional requirements. Clear communication with stakeholders is essential for accurate requirement specifications.
 - Miscommunication can lead to scope creep or mismatched expectations.
2. **Project Success and Risk Management:**
 - Stakeholders influence decision-making processes, timelines, and budget allocation.
 - Ignoring stakeholder input may result in a product that fails to meet expectations, causing dissatisfaction and financial loss.
3. **Continuous Feedback and Agile Development:**
 - Stakeholders are integral to agile methodologies like Scrum, where their feedback shapes iterative improvements.
 - Lack of stakeholder engagement reduces the effectiveness of iterative cycles and can lead to project delays.
4. **Conflict Resolution:**
 - Diverse stakeholder interests often result in conflicts (e.g., budget constraints vs. desired features). Managing these conflicts effectively is crucial for project harmony.
5. **Regulatory and Compliance Issues:**
 - External stakeholders like regulators ensure adherence to legal and ethical standards. Failing to address these concerns can result in penalties or project cancellations.

Stakeholders play a pivotal role in the identification and resolution of "***pain in the process***" and "***unmet needs***." As the primary users or influencers of a software system, stakeholders provide critical insights that shape problem identification and solutions.

1. Stakeholders as Sources of Problems and Insights

Stakeholders as Observers of Pain Points:

- Stakeholders often encounter process inefficiencies directly and can provide firsthand information about challenges.
 - Example: A warehouse manager (stakeholder) might highlight delays caused by a manual inventory system.

What are these?

- A healthcare provider needing faster patient record access.
- A school administrator requesting an automated timetable generator.

Stakeholders as Identifiers of Unmet Needs:

- Stakeholders express expectations that the current system does not fulfill. These gaps reveal unmet needs.
 - Example: A customer might demand a mobile-friendly interface for an e-commerce platform, pointing to an unmet need for accessibility.

2. Stakeholders in Problem-Solving Frameworks

Engagement with Stakeholders:

- Stakeholders must be actively involved in:
 - **Identifying pain points** by sharing their experiences.
 - **Defining unmet needs** through feedback and suggestions.

Feedback Loops:

- Engage stakeholders in continuous feedback loops to validate and refine problem definitions and solutions.
 - Example: An agile development cycle includes stakeholders in sprint reviews to ensure alignment with their priorities.

'How' Strategy:

- Simulate a scenario where learners can act as developers and stakeholders provide iterative feedback.
 - Assign roles such as "business owner," "end-user," and "developer."

3. Connecting Stakeholders to "Pain in the Process"

Case Studies:

- Example: In a retail business, stakeholders (store managers) reveal pain points like slow checkout processes or stock discrepancies. This feedback drives the development of point-of-sale systems or inventory tracking tools.

What Learner Developers Can Do?:

- 1. Interview Stakeholders:** conduct mock interviews with peers playing stakeholders to identify inefficiencies.
- 2. Process Mapping with Stakeholders:** collaborate with "stakeholders" to map workflows and identify bottlenecks.

4. Connecting Stakeholders to "Unmet Needs"

Unmet Needs Discovery via Stakeholders:

- Stakeholders articulate aspirations that current systems fail to deliver.
 - Example: Employees requesting a feature to generate automated reports highlight an unmet need for reporting efficiency.

Use of Surveys and Workshops:

- how to design stakeholder surveys or facilitate workshops to elicit unmet needs.
 - Example Questions:
 - "What feature do you wish existed?"
 - "What tasks are unnecessarily difficult?"

Example:

- how stakeholder input led to innovations like ride-sharing apps addressing unmet needs for affordability and convenience in transportation?

5. Developing Skills to Leverage Stakeholder Input

To bridge the gap between stakeholder feedback and problem-solving, one must develop the following competencies:

- 1. **Active Listening:**
 - Understand stakeholder perspectives without bias.
- 2. **Empathy Mapping:**
 - Teach students to create empathy maps that visualize stakeholders' feelings, challenges, and desires.
- 3. **Requirement Analysis:**
 - Use stakeholder discussions to break down high-level needs into actionable requirements.

Activity:

- Can create a requirement specification document based on stakeholder interviews.

6. End-to-End Process: Stakeholders, Problems, and Solutions

Integrated Approach:

- (1) **Stakeholder Identification:**
 - Teach students to classify stakeholders as primary (e.g., users) or secondary (e.g., regulators).
- (2) **Problem Discovery:**
 - Use stakeholder feedback to identify pain points and unmet needs.
- (3) **Solution Proposals:**
 - Involve stakeholders in brainstorming and validating potential solutions.
- (4) **Implementation and Feedback:**
 - Engage stakeholders in testing prototypes and refining implementations.

Example Framework: Stakeholder-Driven Problem-Solving

Step	Activity	Stakeholder Involvement
Identify Stakeholders	Map out key stakeholders	Stakeholders self-identify their roles and interests
Gather Insights	Conduct interviews and surveys	Stakeholders share pain points and needs
Analyze Problems	Use tools like Root Cause Analysis	Validate analysis with stakeholders
Brainstorm Solutions	Collaborate on solution ideation	Stakeholders contribute ideas
Develop and Test Solutions	Build prototypes; conduct usability testing	Stakeholders provide feedback on iterations
Implement and Review	Roll out solutions and assess impact	Stakeholders evaluate the system post-implementation

Connecting Requirements to Problems

Once we've identified the stakeholders, the next step is to gather **requirements** — a detailed list of what the software system must do to solve the problem. This process is called **Requirements Engineering**, and it forms the foundation of the **Software Process**."

How to Introduce Requirements?

Slide Title: "What Are Requirements?"

Definition of Requirements:

Requirements describe **what the software system must do** to meet the needs of its stakeholders.

Requirement Type	Description	Example
Functional Requirements	Define specific features and functions	"The system must allow students to register for courses."
Non-Functional Requirements	Define system qualities (e.g., performance, security)	"The system must handle 1,000 concurrent users."

Why Does Software Engineering Focus on Requirements?

Software engineering focuses on **requirements** because they define the expectations, constraints, and goals of a software system, serving as the foundation for its design, development, and validation. This emphasis ensures alignment with stakeholder needs, minimizes risks, and optimizes resource utilization throughout the software development lifecycle (SDLC).

Why Requirements Are Critical:

1. **Requirements define what success looks like.**
 - If the system doesn't meet the requirements, it **fails**.
2. **Requirements guide the entire Software Process.**
 - They influence **design, development, and testing**.
3. **Requirements reduce misunderstandings.**
 - They ensure the **stakeholders and developers** are on the same page.

📌 How to start the requirement process?

Activity: Identify Stakeholders and Requirements

Choose a domain (e.g., university portal, hospital management, e-commerce).

1. Identify the key stakeholders.

2. List 3 functional requirements and 2 non-functional requirements.

Example Output:

System	Stakeholder	Functional Requirement	Non-Functional Requirement
University Portal	Students	Students can register for courses	The system must support 500 concurrent users
	Faculty	Faculty can upload course materials	The system must load pages in < 2 seconds

"To build successful software, we need to move beyond merely understanding the problem domain and delve into the challenges faced by the people involved. This requires identifying the stakeholders, uncovering the pain points in their current processes, and addressing the unmet needs they experience. By capturing their requirements with precision and empathy, we can transition into software processes that not only meet but exceed expectations, delivering a system that solves real problems effectively and transforms their experience."

2. Software Processes

*"To build successful software, we need to move beyond merely understanding the **problem domain** and delve into the **challenges faced** by the **people involved**. This requires identifying the **stakeholders**, uncovering the **pain points** in their current processes, and addressing the **unmet needs** they experience. By capturing their **requirements** with precision and empathy, we can transition into **software processes** that not only meet but **exceed expectations**, **delivering a system that solves real problems effectively and transforms their experience**."*

Why Software Processes Matter: A CSO (Concept-Strategy-Operation) Perspective

Story of Building House - Home:

*"Imagine you're leading a team of architects and builders tasked with constructing a custom home for a family. The family has shared their vision: **a spacious living room, energy-efficient windows, and a garden with native plants.** Excited by the project, you jump right into construction without creating a blueprint or assigning specific roles."*

What Happens Next?

1. Confusion Among Team Members:

- Some builders start working on the living room, others on the garden, but there's no coordination.
- Mistakes arise: the living room's walls are built where the garden patio should be.

2. Missed Deadlines:

- The construction team faces delays because no clear milestones were defined.
- When the family visits, they're frustrated because progress doesn't match their expectations.

3. Compromised Quality:

- Without regular checks or feedback from the family, the final design doesn't align with their vision.
- The energy-efficient windows are installed incorrectly, leading to higher costs to fix the error.

The Realization

*"At this point, you realize the problem wasn't the team's skills or the family's vision—it was the absence of a structured **process** to guide the project."*

Reframing with a Process

If a **construction process** had been followed:

1. Concept:

- The family's needs and pain points (e.g., energy efficiency, garden design) would have been clearly documented.

2. Strategy:

- A detailed blueprint would have been created, breaking the project into phases: planning, foundation, structure, interiors, and landscaping.

3. Operation:

- The team would have regular check-ins with the family, ensuring each milestone aligns with their expectations.
- Quality checks (e.g., window alignment tests) would ensure issues are resolved early.

Conclusion

"Processes provide the structure needed to transform abstract visions into successful outcomes, whether you're building a home or software. Without them, even the most skilled teams can lose direction."

Story from Computational Domain:

"Imagine you're part of a team building a new university portal. The requirements are clear: allow students to register for courses, let faculty manage assignments, and ensure administrators can generate reports. You dive in, coding furiously, but soon realize the following problems:"

- **Confusion among team members:** Who handles what?
- **Inconsistent results:** Different modules work differently, confusing stakeholders.
- **Deadline chaos:** The system is delayed, and no one knows why.

"At this point, you realize what's missing: a structured plan that tells everyone not just **what to do**, but also **how to do it**, and **why it's being done**. That's the role of a **software process**."

🌱 Why Software Processes Matter (Framed in CSO)

Concept: What is the Purpose of a Software Process?

A **software process** provides a **framework** to guide teams in delivering high-quality software efficiently and predictably.

• Core Idea:

To **structure the software development journey** into **manageable phases** like planning, development, testing, and delivery.

• Why It's Needed:

- **Unmet Need:** Stakeholders often lack visibility into development progress.
- **Pain in the Process:** Without a process, teams face **delays**, **quality issues**, and **misaligned efforts**.

Strategy: How Does a Software Process Work?

A software process defines **phases, activities, and deliverables** at each step of the development lifecycle. It ensures collaboration and alignment across stakeholders and team members.

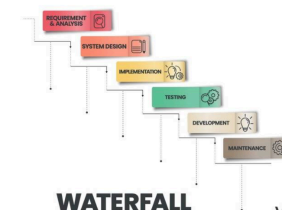
• Approach:

Different strategies suit different needs:

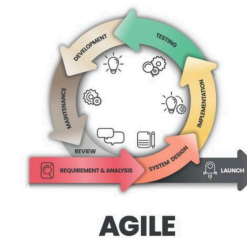
- **Waterfall:** Sequential, suited for fixed requirements.
- **Agile:** Iterative, great for dynamic requirements.
- **V-Model:** Integrates testing with development for high-quality systems.

• Key Elements:

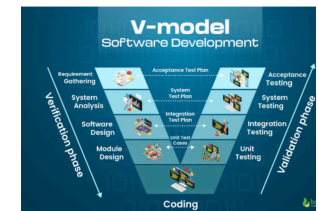
- **Planning:** Clarifies scope and objectives.
- **Execution:** Implements the solution iteratively or sequentially.
- **Validation:** Ensures the software meets the defined requirements.



WATERFALL



AGILE



Operation: How Do We Implement a Software Process?

Operations involve **day-to-day practices** within the process framework, such as:

- **Collaborating with Stakeholders:**
Regularly gather feedback and adjust accordingly (e.g., Agile sprints).
- **Monitoring Progress:**
Use tools like **JIRA** or **Trello** to track tasks and deadlines.
- **Testing and Validation:**
Implement quality checks (e.g., CI/CD pipelines in DevOps).

💡 **Example:**

In an Agile project:

- **Concept:** Deliver working software iteratively.
- **Strategy:** Break the project into sprints, each delivering a small feature.
- **Operation:** Hold daily stand-ups to track progress and address blockers.

🎯 **The Core Why: Ensuring Predictability, Quality, and Collaboration**

Without a Software Process:

- **Lack of Direction:** Teams work inconsistently, leading to wasted effort.
- **Missed Deadlines:** Delays due to unclear milestones.
- **Unhappy Stakeholders:** Software fails to meet needs due to poor collaboration.

With a Software Process:

- **Clear Plan:** Teams know what to do, when, and why.
- **On-Time Delivery:** Timelines are realistic and milestones are clear.
- **Stakeholder Satisfaction:** Iterative feedback ensures needs are met.

🌟 **Conclusion: The CSO Perspective on Software Processes**

*"A software process is the **concept** that structures software development. It provides a **strategy** for organizing activities and ensures effective **operations** that deliver high-quality software efficiently."*

📚 **Managerial and Technical Areas in the CSO Framework for Software Processes**

🔑 Managerial Areas

These areas focus on **planning, coordination, and stakeholder management**.

🌱 Concept (Why It Exists)

Managerial Areas:

- **Stakeholder Alignment:** Ensure all parties agree on the goals of the software process.
- **Problem Framing:** Understand and articulate unmet needs and pain points.
- **Risk Identification:** Anticipate challenges that could impact timelines, quality, or cost.

🌱 Strategy (How It Works)

Managerial Areas:

- **Process Selection:** Choose the appropriate process model (e.g., Agile for dynamic projects, Waterfall for fixed scope).
- **Resource Management:** Allocate team members, tools, and budgets efficiently.
- **Continuous Feedback:** Establish mechanisms for iterative feedback from stakeholders.

🔧 Technical Areas

These areas involve implementing and optimizing the software process through engineering skills.

🌱 Operation (What We Do)

Technical Areas:

- **Collaboration Tools:** Use platforms like JIRA, Trello, or Azure DevOps for task management.
- **Testing and Validation:** Implement automated testing frameworks like Selenium or JUnit.
- **Continuous Integration and Deployment (CI/CD):** Set up pipelines to automate build, test, and deployment cycles.

🌟 Soft Skills for Managerial Areas

Soft Skill	Why It's Important
Communication	To articulate goals, manage expectations, and handle conflicts among stakeholders.
Empathy	To understand stakeholders' perspectives, addressing unmet needs effectively.
Time Management	To prioritize tasks and ensure project deadlines are met.
Adaptability	To pivot strategies in response to changing requirements or unforeseen challenges.
Leadership	To motivate the team, resolve issues, and maintain focus on goals.
Negotiation	To balance trade-offs between scope, quality, and deadlines.

📚 Examples of Combined Managerial and Technical Roles

1. Stakeholder Requirement Gathering (Managerial)

- **Managerial:** Conduct interviews and facilitate discussions to define requirements.
- **Technical:** Translate these requirements into technical specifications using tools like UML or BPMN.

2. Risk Mitigation (Managerial)

- **Managerial:** Identify potential risks in the process, such as resource constraints.
- **Technical:** Use predictive analytics tools (e.g., Tableau, Power BI) to simulate scenarios and manage risks.

3. Feedback Incorporation (Managerial)

- **Managerial:** Collect and prioritize stakeholder feedback.
- **Technical:** Adjust system functionality and ensure version control using Git.

🌟 **Conclusion: Balancing Managerial and Technical Roles**

A **software engineer** is not just a coder but a well-rounded professional who must:

- Navigate **managerial tasks** with empathy, leadership, and adaptability.
- Excel in **technical responsibilities** by mastering tools and methodologies.

Thank You

CSL308 SE S26

Software System Classification and Its Role in Modern Software Engineering

Anil Mokhade 13 Feb 2026

Story: “Three Software Teams”

Three software teams graduate from VNIT.
All three teams are equally talented.
All three use modern tools.
All three know Python, Java, Git, Docker, and even some AI.

Team 1 – The Startup App Team

They build a **food delivery mobile app**.

- Users place orders.
- Payments happen online.
- Database stores transactions.
- Servers run in cloud.

They follow Agile.

They release every two weeks.

If a bug comes, they patch it next day.

Worst-case failure?

- Order delayed.
- Customer angry.
- Refund issued.

Business loss.

But no one dies.

Team 2 – The Hospital Device Team

They build software for an **ICU patient monitoring device**.

- Real-time heart-rate tracking.
- Oxygen monitoring.
- Alarm triggers.
- Embedded controller.

They try to follow the same process as Team 1.

Fast releases.

Frequent updates.

Quick patches.

One night, due to a small timing bug,
the alarm does not trigger on time.

A patient’s condition worsens.

Now the failure is not:

- A refund.
- Or a bad review.

It is a life.

Team 3 – The AI Loan Approval Team

They build an AI system for a bank.

- It predicts credit risk.
- Approves or rejects loans.
- Uses machine learning.

System works fine during testing.

After deployment,
economic conditions change.

Model drifts.

Suddenly:

- Good customers are rejected.
- Bad customers are approved.

No crash.

No exception.

No red error message.

But financial damage occurs silently.

What is common in all three?

- All are software?
- All use programming?
- All use databases or models?

“Can we develop all three systems using the same software engineering approach?”

This is why **software systems are classified**.

Not all software systems are equal.

Some are:

- Transaction systems
- Embedded systems
- Safety-critical systems
- AI decision systems

Classification tells us:

- What process to follow
- What testing rigor is required
- What architecture to design
- What risk tolerance is acceptable

AI Twist:

“In classical software, failures were visible.”

- Crash
- Exception
- Timeout

But in AI systems,
failures may be invisible.

The system runs.

But decisions degrade.

That is why today,
classification must also consider:

- Learning behavior
- Autonomy
- Risk impact

What can be concluded from this story?

“Software engineering is not about writing code.
It is about understanding what kind of system you are building.”

“Today we will see how software systems are classified — and why that matters even more in the AI era.”

In **Ian Sommerville**’s classic textbook **Software Engineering**, software systems are broadly classified to help understand **engineering requirements, constraints, risks, and development processes**.

Though editions vary slightly, the commonly discussed categories include:

Stand-Alone Applications

Characteristics:

- Run locally on a PC/mobile device
- No continuous network dependency
- User-centric interaction
- Example: Word processors, photo editors

Engineering Focus:

- UI/UX design
- Performance on local hardware
- File management
- Maintainability

Interactive Transaction-Based Systems

Characteristics:

- Multi-user, concurrent access
- Backend database driven
- ACID transactions
- Security critical

Examples:

- Online banking
- E-commerce platforms
- Hospital management systems

Engineering Focus:

- Data consistency
- Scalability
- Cybersecurity
- Fault tolerance

Embedded Control Systems

Characteristics:

- Software controls hardware
- Real-time constraints
- Safety-critical

Examples:

- Automotive ECUs
- Industrial control
- Medical devices

Engineering Focus:

- Deterministic timing
- Reliability
- Hardware-software co-design
- Safety certification

Batch Processing Systems

Characteristics:

- No direct user interaction
- Process large datasets
- Scheduled jobs

Examples:

- Payroll systems
- Data warehousing jobs

Engineering Focus:

- Throughput optimization
- Data integrity
- Fault recovery

Entertainment Systems

Characteristics:

- User engagement focus
- Performance + graphics intensive
- Rapid evolution

Engineering Focus:

- Real-time rendering
- UX psychology
- High-performance computing

Modeling & Simulation Systems

Characteristics:

- Mathematical models
- Scientific computation
- Predictive capability

Examples:

- Climate simulation
- Flight simulators
- Digital twins

Engineering Focus:

- Numerical stability
- Computational efficiency
- Model validation

Data Collection & Monitoring Systems

Characteristics:

- Real-time sensor input
- Alert generation
- Event-driven

Engineering Focus:

- Streaming architectures
- Reliability
- Low latency

System of Systems

Sommerville's "application type" classification is often presented as a pragmatic set of common software system types. **SoS is sometimes treated separately** (more in systems engineering literature) because it's not just an "application type"; it's a **composition/scale-of-integration type**: a way systems are *organized* in the real world.

Where "System of Systems" fits

A **System of Systems** is a collection of **independent systems** that:

- have their **own owners/management**, budgets, and evolution cycles,
- can **operate independently**, yet
- when connected, produce **new emergent capabilities**.

Examples you can instantly relate to:

- **Smart city** (traffic, policing, utilities, emergency response, citizen apps)
- **UPI ecosystem** (banks + NPCI rails + merchant apps + fraud systems + telecom + KYC services)
- **Airport/Metro operations** (ticketing, signaling, crowd control, surveillance, announcements, apps, payments)
- **Healthcare ecosystem** (HIS, labs, radiology, insurance, e-pharmacy, government portals)

So, even if we keep the earlier Sommerville categories (stand-alone, transaction, embedded, monitoring, simulation, etc.), **SoS sits "above" them** as an integration reality: many of those types coexist inside a larger SoS.

Layer A: Application/System Type (Sommerville-style)

- Stand-alone
- Transaction-based
- Embedded control
- Batch processing
- Entertainment
- Modeling & simulation
- Data collection/monitoring

Layer B: Integration Scale (SoS-aware)

- Single system
- Distributed system / platform
- **System of Systems**

This is a very SE-friendly framing: it directly affects **requirements, architecture, testing, maintenance, and process**.

Why SoS matters even more in the AI-agent era

AI agents don't just "run inside one app". They often **span multiple systems**:

- A "travel agent" AI touches: calendars + email + airline booking + payments + identity verification + customer support ticketing.
- A "campus admin agent" touches: ERP + hostel + mess + library + HR + file systems + approvals.

So, AI agents are frequently **SoS orchestrators**.

New failure modes (great for teaching)

In a single system, failure is often a bug.

In an SoS, failure is often:

- **interface mismatch**
- **policy mismatch** (who is allowed to do what)
- **timing/latency mismatch**
- **data semantics mismatch** (same term means different things in two systems)
- **feedback-loop amplification** (system A reacts to B, B reacts to A → instability)

AI adds:

- **tool misuse** (agent calls wrong API)
- **hallucinated actions** (agent confidently triggers an incorrect workflow)
- **drift in decision quality** (agent behavior changes after model/tool updates)
- **accountability blur** (“who made the decision?”)

Product vs Service perspective

Product companies (building platforms)

SoS thinking drives:

- API strategy, versioning, backward compatibility
- observability across services (tracing, audit logs)
- “governed autonomy” for agents (permissions, approvals, guardrails)
- multi-tenant risk management

Service companies (building/ integrating for clients)

SoS thinking drives:

- integration architecture + contracts + SLAs
- stakeholder alignment across departments/vendors
- staged rollouts, fallback plans, incident response
- compliance and auditability

“System of Systems: the dominant reality of modern software”

- Not an application category; an *integration class*
- Contains multiple application types
- Key issues: interoperability, evolution mismatch, emergent behavior, governance
- AI agents accelerate SoS interactions → higher value + higher risk

Thank You

CSL508 SE S26

Software Process Models

Anil Mokhade 17 Feb 2026

“Which is more difficult — writing code or deciding what to code?”

Writing code is the easiest part of software engineering.

*The hardest problem in software development is not how to build it.
It is deciding what exactly to build.*

The Core Premise

WHAT to build?

vs

HOW to build?

“Software development failures rarely happen because programmers cannot code.
They happen because the wrong system was built.”

A company builds a campus attendance system.

They perfectly implement:

- Biometric authentication
- Reporting
- Admin dashboards

Everything works technically.

But faculty say:

- It doesn't match real classroom workflows.
- It doesn't handle guest lectures.
- It creates administrative overhead.

Technically correct.

Conceptually wrong.

Why?

Because requirement understanding was shallow.

Why “What to Develop?” Is the Toughest Question Stakeholders Don't Know What They Want

Users often say:

- “Make it simple.”
- “Make it fast.”
- “Make it smart.”

But what does that mean technically?

Requirements Are Often Implicit

- Business rules are undocumented.
- Workflows are informal.
- Exceptions are not described.

What is a 'Workflow'?

A piece of work passed through:

- Sequencing of Industrial, Administrative or other processes
- From initiation to completion
- Helps in understanding of data-dependency
- Removal of flow dependency
- Flexibility
- Reusability
- Integration
- Scalability

Requirements Change with Understanding

Once users see a prototype, they say:

“Oh, this is not what I meant.”

AI Era Makes It Worse

Now users say:

“Add AI.”

But:

- What kind of AI?
- What accuracy?
- What risk tolerance?
- What data quality?

What 'Deep Insight' we can draw

“Coding errors are visible.

Requirement errors are invisible — until it is too late.”

Classification told us:

- What kind of system we are building.
- What risk level it carries.

But classification does NOT automatically define:

- Exact system boundaries.
- Detailed functionality.
- Non-functional constraints.
- Stakeholder expectations.

That is requirement understanding.

Six Generic SDLC Steps

- 1.Requirement Understanding
- 2.Requirement Analysis
- 3.Design
- 4.Coding
- 5.Testing
- 6.Deployment & Maintenance

The first two steps determine the quality of all the remaining four.

If requirement understanding is weak:

- Design is misaligned.
- Coding solves wrong problem.
- Testing validates wrong functionality.
- Maintenance becomes chaotic.

Conceptual Clarification (Very Important)

Requirement Understanding $\neq$ Requirement Documentation

Requirement Understanding means:

- Understanding problem context.
- Understanding stakeholders.
- Understanding constraints.
- Understanding risks.
- Understanding environment.

It is about problem framing.

Before we discuss how to build software through different process models, we must first accept that defining what to build is the most intellectually demanding part of software engineering.

Process models exist to structure and control this difficult activity of requirement discovery.

The cost of building the wrong system correctly is higher than building the right system slowly.

Final Insight

In today's AI world, where tools can generate code instantly:

The true engineering skill is:

- Asking precise questions
- Identifying ambiguity
- Defining constraints
- Clarifying objectives

Not typing syntax.

Structured thinking under uncertainty.

Story: The Shortcut

A startup hired two fresh engineering teams.

Both teams were given software projects.

Team 1 was asked to build a **movie ticket booking app**.

Team 2 was asked to build software for a **blood pressure monitoring device**.

Both teams were smart.

Both teams knew the latest tech stack.

Both teams said:

"Let's move fast. We will improve later."

Team 1 launched in 2 weeks.

Some bugs appeared.

Users complained.

They patched it in the next release.

Business continued.

Team 2 also rushed.

During field testing, the device sometimes delayed sending alerts.

The software did not crash.

It just delayed by a few seconds.

But those few seconds mattered.

The project was stopped.

Investors withdrew.

The hospital refused certification.

What was the difference?

- Risk?
- Impact?
- Consequences?
- Safety?

Both were software.

Both were coded in similar languages.

But the consequences of failure were completely different, weren't it?

That difference changes how we should build the system.

Last topic we classified software systems.

Now, we will see why different systems require different development discipline.

That discipline is defined by **software process models**.

Both were software. But the consequences of failure were different.

We will now understand why classification alone is not enough.

Food App
ICU Device
AI Loan System

If this fails, what happens(consequences), we now know.

The difference in consequences is called **risk**.

$\text{Risk} = \text{Probability} \times \text{Impact}$

Different system types carry different levels of risk.

Why Risk Should Come Before Process Models

We have already learnt:

Classification of software systems

Now students understand:

- Embedded systems
- Transaction systems
- AI systems
- SoS systems

At this moment, they are ready to ask:

Why can't we use the same development approach for all?

That question opens the door to risk.

Because:

Different systems → Different consequences of failure → Different risks

So risk becomes the bridge.

From Classification to Risk

"If this embedded system fails, what happens?"

"If this food delivery app fails, what happens?"

"If this AI loan system fails, what happens?"

What are your answers:

- Annoyance?
- Financial loss?
- Safety hazard?
- Ethical damage?

Can we then say:

Failure Impact = Risk?

Do we realize the need for discipline?

From Risk to Process

So, if we say:

"If risk levels differ, should development discipline differ?"

This is where the Software Processes come in:

"Software process models are structured ways of managing risk and uncertainty."

Do process models make sense?

“Today we study Waterfall, V-Model, Spiral...”

- Another theoretical topic?
- Just exam content?
- Sequence of steps to memorize?

Let us understand why process models exist.

Why Not Start With Formal Risk-Driven Theory?

If we immediately start with:

“Risk-driven development, Barry Boehm...”

It becomes abstract.

Better approach:

Let us discover risk intuitively through consequences.

Then formalize it.

“If risk levels differ, should our development discipline be the same?”

No.

System Type → Risk Level → Required Discipline

We need a structured way to manage that discipline.

“That structured discipline is called a **software process model**.”

The Correct Order

Classification



Failure Consequences



Risk Awareness



Need for Structured Approach



Process Models



Risk-Driven Model (Spiral) as special case

This sequence builds conceptual clarity.

Process model is not the next engineering step after classification.

It is not:

Classification → Choose Process → Then Define Requirements.

Instead...”

Process Model

= Framework for

Requirement Discovery & Risk Control

“Process model defines how we will discover, refine, validate, and control requirements.”

Let us see the analogy:

“Process model is like the rules of a scientific experiment.

Requirements are what we discover during that experiment.”

“This means requirements engineering happens inside the process model.”

This is the conceptual correction.

How Different Processes Handle Requirements Differently

Let us introduce process models briefly but through the lens of requirements.

◆ Waterfall

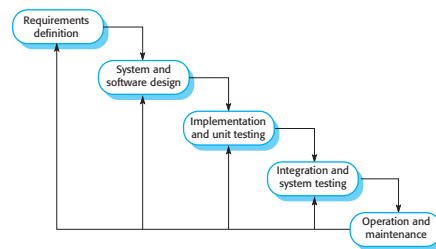
Assumption:

Requirements are stable.

Approach:

- Define requirements fully upfront.
- Freeze them.
- Proceed sequentially.

“This works when uncertainty is low.”



V-Model

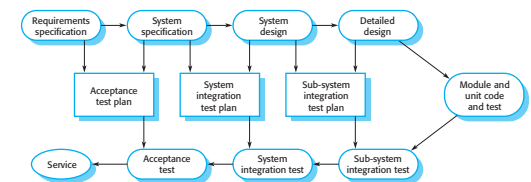
Assumption:

Failure cost is high.

Approach:

- Define requirements carefully.
- Plan testing alongside requirements.
- Strong verification.

“Used when safety risk is high.”

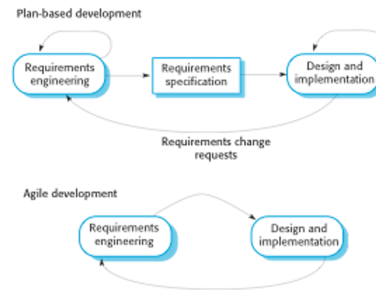
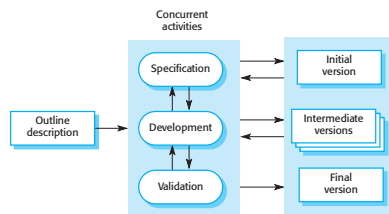


Agile

Assumption:
Requirements evolve.

Approach:

- Discover requirements incrementally.
- Continuous user feedback.
“This works when uncertainty is high but failure impact is moderate.”



Spiral (Risk-Driven)

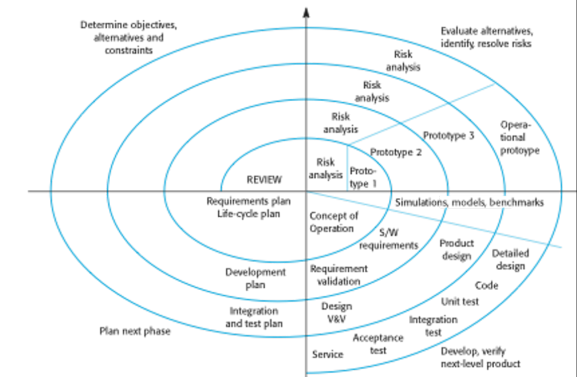
Barry Boehm

Assumption:
Major risks exist.

Approach:

- Identify biggest risk.
- Prototype to reduce it.
- Refine requirements based on learning.

“Here risk directly shapes requirement understanding.”



AI Era Clarification

“In AI systems, requirements are not just functional.

They include:

- Accuracy threshold
- Bias limits
- Data quality
- Monitoring needs.”

Thus process must allow:

- Iteration
- Validation
- Monitoring
- Refinement

Traditional: Requirements → Build

AI Systems: Requirements ↔ Data ↔ Model ↔ Validation (loop)

“In AI systems, requirements may evolve after deployment.”

Where Risk-Driven Approach Comes In

We have seen Waterfall and V-model briefly, now can we say:

“These models assume requirements stability?”

The Spiral Model by

Barry Boehm

Spiral explicitly organizes development around identifying and reducing risks.

The spiral model is like evolution, not just another diagram.

System Type
→ Failure Impact
→ Risk Level
→ Required Discipline
→ Process Model
This creates logical coherence.

“Different systems fail differently.
And when failure consequences change, development discipline must change.”

What Is a Risk-Driven Approach?

A **risk-driven approach** means:

We organize software development activities based on identifying and reducing the highest risks first.

Instead of asking:

- “What should we build next?”

We ask:

- “What can go wrong?”
- “What uncertainty can damage this project?”
- “What failure would be most costly?”

Then we design the development plan around reducing those risks early.

Simple Classroom Definition

A risk-driven process prioritizes solving the most dangerous uncertainties before investing heavily in full development.

What Is “Risk” in Software Engineering?

Risk = Probability of failure × Impact of failure

Types of risk:

1. Technical risk (Will this technology work?)
2. Requirement risk (Do we really understand the problem?)
3. Schedule risk (Can we finish on time?)
4. Performance risk (Will it scale?)
5. Safety risk (Can it cause harm?)
6. AI risk (Bias, drift, unpredictability)

Where Did Risk-Driven Thinking Come From?

The Spiral Model by

Barry Boehm

introduced risk as the central organizing principle of development.

Unlike Waterfall (sequence-driven),

Spiral is risk-driven.

How Is It Different From Traditional Approach?

Traditional (Activity-Driven)

Requirements → Design → Code → Test

Activities are fixed in order.

Risk-Driven

Identify Risk → Prototype/Test → Reduce Risk → Then Continue

The order depends on uncertainty.

Example 1 (Simple UG-Level Example)

You are building a campus payment system.

Biggest risk?

Security breach.

Risk-driven approach:

- First test authentication & encryption.
- Perform security prototype.
- Conduct threat modeling.
- Then build full features.

Not the other way around.

Example 2 (AI System)

You are building AI for loan approval.

Biggest risk?

Model bias & incorrect decisions.

Risk-driven approach:

- First evaluate training data.
- Prototype model.
- Test fairness.
- Validate on unseen data.
- Then deploy.

Example 3 (Embedded System)

You are building real-time traffic signal controller.

Biggest risk?

Timing failure.

Risk-driven approach:

- Test real-time constraints first.
- Validate hardware integration.
- Then build UI dashboard later.

Examples of Common Project, Product, and Business Risks

- Staff Turnover
- Management Change
- Hardware Unavailability
- Requirement Change
- Specification Delays
- Size Underestimated
- CASE Tool underperformance
- Technology Change
- Product Competition

Why Risk-Driven Approach Is Important Today

Because systems today are:

- Large
- Distributed
- AI-enabled
- Highly interconnected (System-of-Systems)

Failure cost is high.

Ignoring risk leads to:

- Technical debt
- System instability
- Ethical failure
- Legal liability

Strategy A:

Build everything → Then test.

Strategy B:

Find biggest danger → Solve it first → Then build rest.

Which one is more intelligent?

That is risk-driven thinking.

Concept

Risk = uncertainty with potential loss.

Strategy

Development sequence is organized around reducing highest risks first.

Operation

Activities include:

- Prototyping
- Simulation
- Proof-of-concept
- Early validation
- Iterative review

“Good engineers don’t just build features.
They eliminate dangers early.”

In AI Era

Risk-driven approach becomes even more important because:

- AI systems behave probabilistically.
- Failures are often silent.
- Bias can cause social harm.
- Model drift can degrade performance.

So iterative validation and monitoring are forms of risk control.

Risk-driven development means structuring the software process around identifying, analyzing, and mitigating the most critical risks early in the lifecycle.

Where Does Process Model Sit in the Big Picture?

simplified hierarchy:

Problem Context



System Classification



Risk & Uncertainty Assessment



Process Model Selection



Requirements Engineering (within that model)

Notice:

Process model shapes how requirements are handled.

How Process Model Helps in Defining the Problem

This is the key answer to this question.

Process models influence:

1. How requirements are elicited
2. How uncertainty is handled
3. How stakeholders are involved
4. How validation happens
5. How risk is reduced

Let us examine concretely.

Example 1: Waterfall Model

If you adopt Waterfall:

- You assume requirements are stable.
- You attempt complete requirement specification upfront.
- You emphasize documentation.
- You minimize requirement changes later.

So the process model forces:

Clear, stable, well-documented problem definition early.

Thus process model shapes problem framing strategy.

Example 2: Spiral Model

(Barry Boehm)

Spiral assumes:

- Requirements are uncertain.
- Risks are high.
- Prototyping is necessary.

So the process encourages:

- Iterative problem refinement.
- Risk-based requirement discovery.
- Early validation through prototypes.

Here, the process explicitly acknowledges that:

The problem is not fully understood initially.

Example 3: Agile

Agile assumes:

- Requirements evolve.
- Users learn as system evolves.

So requirement definition becomes:

- Incremental
- User-feedback driven
- Adaptive

Thus the process model determines how we define and refine the problem.

In AI-Based Systems

This becomes even more important.

AI systems often have:

- Unclear performance boundaries.
- Data uncertainty.
- Ethical concerns.
- Unknown failure modes.

If you choose Waterfall blindly:

- You may freeze requirements prematurely.
- You may ignore model uncertainty.

If you choose iterative + MLOps:

- You incorporate validation loops.
- You continuously refine problem understanding.

Thus process model determines:

How we manage ambiguity in the problem.

“Would you build a bridge first and test whether it stands...
or test a small model first?”

What Is a Prototype? (Simple Definition)

A prototype is a preliminary version of a system built to explore, validate, or reduce uncertainty before full-scale development.

Underline:

Explore. Validate. Reduce Uncertainty.

Prototype is not primarily for coding practice.

It is for learning.

Why Prototype Appears in Process Discussion

When do we prototype?

- When requirements are unclear
- When risk is high
- When technology is new
- When stakeholders cannot visualize the system
- When AI model performance is uncertain

So prototype is a **risk-reduction tool**.

Where Prototype Fits in Software Process

In Waterfall:

- Rare
- Usually before requirement freeze

In Agile:

- Frequent
- Small increments

In Spiral:

- Core mechanism

In AI:

- Almost mandatory

“Prototype is a learning instrument, not a production system.”

Why This Matters in AI Era

Because AI tools now allow:

- Rapid prototyping
- Code generation instantly
- Model experimentation quickly

So engineers must learn:

- When to prototype
- What to prototype
- What risk they are trying to reduce

Otherwise they just generate random systems.

Throwaway Prototype

Purpose:

- Understand requirements
- Get user feedback
- Explore idea

After learning, it is discarded.

Example:

- UI mockup
- Early AI model experiment

In Risk-Driven Process (Spiral)

Mention briefly:

Barry Boehm

Spiral model uses prototyping to:

- Identify major risk
- Build prototype to reduce it
- Evaluate
- Move forward

So prototype is central to risk-driven development.

Evolutionary Prototype

Purpose:

- Start small
- Gradually refine
- Eventually become final system

Used often in Agile and AI development.

In AI Systems (Very Important Today)

Prototype becomes even more critical because:

- We don't know model accuracy beforehand.
- Data quality is uncertain.
- Performance may vary.
- Bias must be checked.

So AI projects often begin with:

- Proof-of-concept model
- Small dataset experiment
- Feasibility validation

That is prototyping.

“Whenever uncertainty is high, prototyping becomes intelligent.
Whenever risk is high, prototyping becomes necessary.”

Conceptual Clarification (Very Important)

Students often think:

Requirements → then process → then development.

Actually:

Process model is the *meta-structure* that governs how requirements evolve.

Think of it this way:

Requirements are content.

Process is the container.

Without container, content spills everywhere.

Concept

Process model = structured approach to managing uncertainty, change, and risk in software development.

Strategy

Different system types demand different strategies:

Stable, regulated system → structured upfront requirement strategy (Waterfall/V-model)

Uncertain, evolving system → iterative requirement discovery (Spiral/Agile)

AI-driven system → continuous validation and retraining (MLOps)

Thus process model influences:

- Requirement completeness
- Stakeholder involvement
- Risk control

Operation

Operationally, process determines:

- When requirements are frozen.
- How changes are handled.
- How validation is performed.
- How feedback loops are integrated.

Analogy

Let us reaffirm:

The process model is like the rules of a scientific experiment.

Requirements are like hypotheses.

Without experimental design, hypotheses remain vague.

Process gives structure to discovery.

Key Insight

Process model does not define the problem.

It defines how rigorously and adaptively we discover the problem.

That is the subtle but critical distinction.

In Today's AI World

AI makes problems more ambiguous.

Therefore:

- Iterative validation becomes essential.
- Risk-driven thinking becomes central.
- Continuous monitoring becomes part of problem definition.

Thus process models are more important today, not less.

Conclusion

“Process models are not about coding order.

They are about managing uncertainty in understanding the problem.”

This sentence alone clarifies everything.

Thank You